directly consumes the OpenAI API. `self.client` is an OpenAI client created earlier during class initialization, and the rest of the code you've seen used in earlier examples.

Listing 7.10 `oai_agent.py` (`get_response`)

```
async def get_response(self, user_input, thread_id=None):
    self.messages += [{"role": "user",
                       "content": user_input}]      #1
    response = self.client.chat.completions.create(     #2
        model=self.model,
        messages=self.messages,
        temperature=0.7,      #3
    )
    self.last_message = str(response.choices[0].message.content)
    return self.last_message     #4
```

#1 Adds the user_input to the message stack
#2 The client was created earlier and is now used to create chat completions.
#3 Temperature is hardcoded but could be configured.
#4 Returns the response from the chat completions call

Like the agent profiles, Nexus uses a plugin system that allows you to place new agent engine definitions in the `nexus_agents` folder. If you create your agent, it just needs to be placed in this folder for Nexus to discover.

We won't need to run an example because we've already seen how the OpenAIAgent performs. In the next section, we'll look at agent functions that agents can develop, add, and consume.

7.5 Giving an agent actions and tools

Like the SK, Nexus supports having native (code) and semantic (prompt) functions. Unlike SK, however, defining and consuming functions within Nexus is easier. All you need to do is write functions into a Python file and place them into the `nexus_ actions` folder.

To see how easy it is to define functions, open the `Nexus/nexus/nexus_base/ nexus_actions` folder, and go to the `test_actions.py` file. Listing 7.11 shows two function definitions. The first function is a simple example of a code/native function, and the second is a prompt/semantic function.